Data Structures and Algorithms -3 (25CS2103E)

Project - Warehouse Inventory System

Team Members -

Santhosh Pendyala-2520030330
Haadi Hasil-2520030297

Sashank Chivukula-2520030405

Under the Esteemed Guidance of

DR. V Sireesha

Contents

- Abstract
- Introduction
- Literature survey
- Problem Statement and Objectives
- Hardware and Software requirements
- Results
- Conclusion
- Future Scope

Abstract

The Warehouse Inventory System is an algorithm-driven application built to manage product inventory and supplier coordination across large-scale warehouses efficiently. It tackles three core challenges — typo-tolerant product search, cost-optimal restocking, and supplier coordination — using string-matching (KMP, Rabin-Karp) for exact SKU/name search, Edit Distance (Wagner-Fischer) for fuzzy search, dynamic programming (0/1 Knapsack) for budget-optimal restocking, and network-flow algorithms (Ford-Fulkerson, Edmonds-Karp) for supplier-to-warehouse flow.

Supplier relationships are modeled as a flow network, enabling max-flow / min-cut analysis to find bottleneck suppliers and bipartite matching for optimal assignment. NP-hard order scheduling is handled with approximation algorithms, and reservoir sampling powers randomized stock audits. By combining string algorithms, DP, network flow, and randomized techniques, the system replaces linear search and manual tracking with efficient, scalable algorithmic solutions.

Introduction

Managing inventory in a large warehouse gets messy fast. With thousands of products, exact-match search fails on typos, restocking decisions ignore budget/space limits, and supplier coordination is handled manually — and it only gets worse as inventory grows. This project sets out to fix that.

The issue: Manual, exact-match search and tracking don't scale or tolerate typos in large inventories.

The impact: Leads to missed matches, wasteful restocking, and slow, bottlenecked supplier coordination during shortages.

The goal: Enable typo-tolerant product search, cost-optimal restocking, and bottleneck-free supplier coordination.

The approach: Use string-matching and edit-distance algorithms for search, dynamic programming for restocking, network-flow algorithms for supplier coordination, approximation algorithms for NP-hard scheduling, and randomized sampling for audits.

The outcome: A faster, more reliable, and provably efficient inventory system that scales smoothly.

Literature Survey

Study/System	Approach Used	Limitation
Traditional WMS (SQL-based)	Relational DB queries (LIKE-based text search)	Slow substring search; no tolerance for typos or misspelled SKUs
Basic inventory scripts	Naive pattern matching, O(n·m)	Doesn't scale to large product catalogs or fuzzy queries
RFID-based tracking systems	Hardware tracking + DB	No algorithmic flow-optimization or provable scheduling guarantees

Problem Statement and Objectives

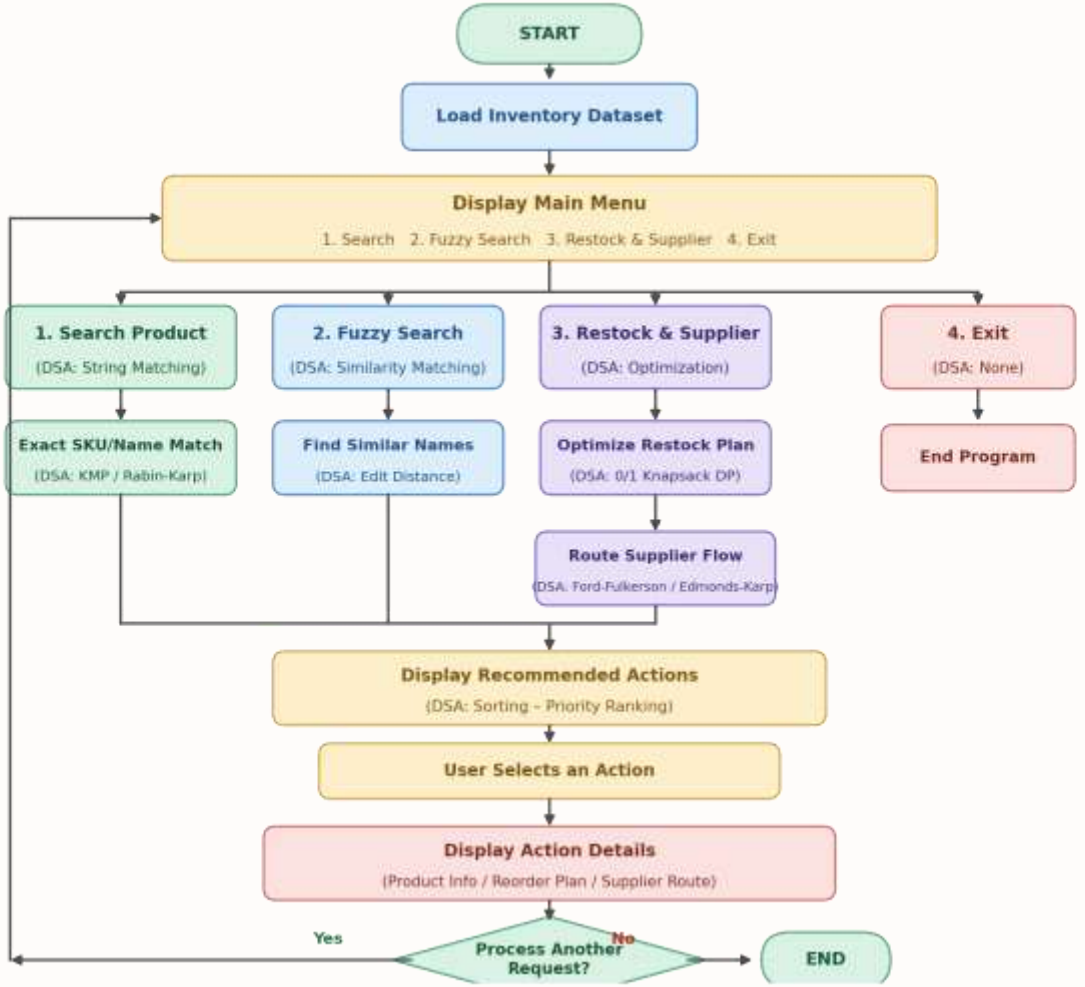
Current warehouse inventory systems often face:

- Slow, exact-match-only product searching
- Sub-optimal restocking under budget/space constraints
- Inefficient supplier coordination
- No handling of NP-hard scheduling or randomized audits

The proposed Functionalities are :

- Product search — exact (KMP/Rabin-Karp) and fuzzy (Edit Distance) matching
- Optimal restocking using Dynamic Programming (0/1 Knapsack)
- Supplier flow-network modelling with Ford-Fulkerson/Edmonds-Karp
-
- NP-hard scheduling via approximation, verified by randomized audits

Flow Chart


START
Load Inventory Dataset
Display Main Menu
1. Search 2. Fuzzy Search 3. Restock & Supplier 4. Exit
1. Search Product
(DSA: String Matching)
2. Fuzzy Search
(DSA: Similarity Matching)
3. Restock & Supplier
(DSA: Optimization)
4. Exit
(DSA: None)
Exact SKU/Name Match
(DSA: KMP / Rabin-Karp)
Find Similar Names
(DSA: Edit Distance)
Optimize Restock Plan
(DSA: 0/1 Knapsack DP)
End Program
Route Supplier Flow
(DSA: Ford-Fulkerson / Edmonds-Karp)
Display Recommended Actions
(DSA: Sorting – Priority Ranking)
User Selects an Action
Display Action Details
(Product Info / Reorder Plan / Supplier Route)
Process Another Request?
Yes
No
END

Results

Operation	Traditional (linear / manual)	Implemented (this project)	Improvement
Exact product lookup by SKU/name	O(n·m)	O(n+m) — KMP / Rabin-Karp	Linear, single pass
Fuzzy / typo-tolerant search	Not available	O(n·m) — Edit Distance (Wagner-Fischer)	New capability
Optimal restocking under constraints	Manual / greedy	O(n·W) — 0/1 Knapsack DP	Provably optimal
Max procurement flow across suppliers	Manual allocation	$O(V \cdot E^2)$ — Edmonds-Karp	Automated, capacity-aware
NP-hard order scheduling	Not available	2-Approximation (matching-based)	New capability, bounded ratio

Hardware and Software requirements

Hardware requirements：

- Processor: Intel i3 / AMD Ryzen 3 or higher (dual-core minimum)
- RAM: 4 GB minimum (8 GB recommended for large dataset testing)
- Storage: 500 MB free disk space
- Standard I/O devices: keyboard, mouse, monitor

Software requirements：

- Operating System： Windows 10/11
- Programming Language： Java
- Git
- Visual Studio Code

Conclusion

- The Warehouse Inventory System successfully demonstrates how advanced algorithms — string matching, dynamic programming, network flow, NP-hard approximation, and randomized sampling — can solve real-world inventory and supply-chain problems efficiently.
- Replacing exact-match search and manual tracking with these techniques significantly improves search flexibility, restocking optimality, and supplier-network coordination.
- The system scales well as inventory size and supplier-network complexity grow, making it suitable for large-scale warehouse operations.
- This project reinforces practical understanding of advanced DSA-3 concepts — string algorithms, dynamic programming, network flow, and randomized/approximation techniques — by applying them to a cohesive, real-world use case rather than isolated exercises.

Future Scope

- Suffix automaton / suffix array indexing — faster multi-pattern and substring search across the catalog.
- GUI/dashboard — visual interface for staff to view stock, flow, and scheduling alerts.
- Min-cost max-flow — optimal-cost procurement instead of max-flow-only routing.
- Dinic's algorithm — faster max-flow computation for larger supplier networks.
- Predictive restocking — forecast demand using historical trends to refine the DP model.
- Parallel processing — parallel reduce/prefix-sum for concurrent, multi-warehouse audits.

(DEEMED TO BE UNIVERSITY)

Thank you